

Today's content

1. Programming paradigms (pattern)
procedural programming
2. Need of object oriented prog
3. Principle and pillars of OOP.

Different types of programming paradigms (patterns)

- 1. procedural → very old / famous prog
- 2. OOP → most commonly used
- 3. functional → will be covered in 'java 8'
- 4. reactive

Java it's procedural, OOP, functional

Python it's procedural, OOP, functional, reactive

C it's procedural language.

Procedural programming

Procedure → "It's a set of instructions".

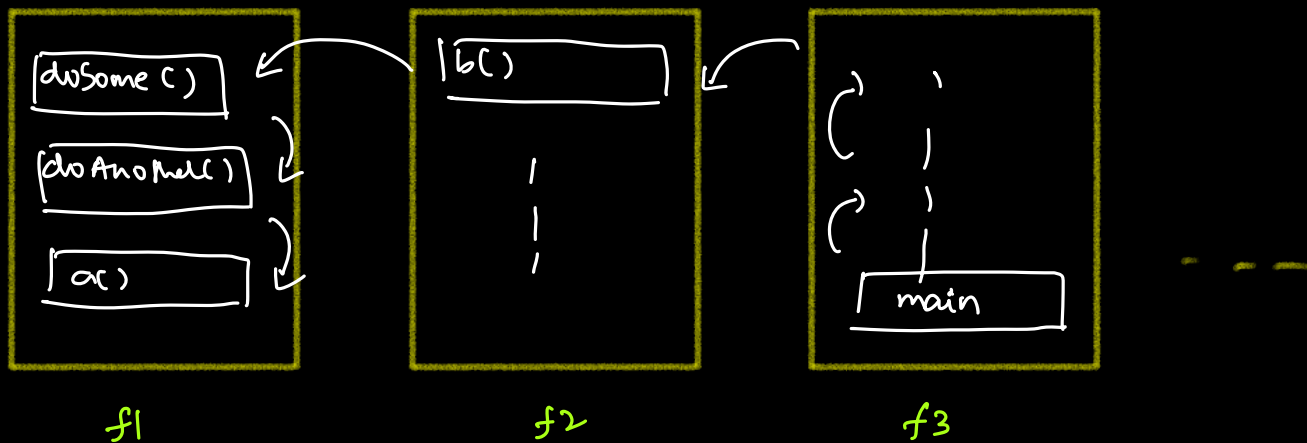


method / function

"procedure is a old age name for method".

In procedural programming,

codebase is structured into multiple methods, with each one of them having their own responsibility, execution of these methods will be triggered by another special method => **main**



This type of code we've written in java also, it also supports.

Problems in prog. languages that only supports procedural paradigm.

Q: What's going on in class?...

- (i) we are learning
- (ii) Keerthi is teaching
- (iii) scaler class is happening.

Basic english.

Subject + Verb

This is how we're supposed to write code, But we cannot do this in procedural programming.

Procedural world.

Task: Print student details

```
printStudent ( name, age, gender)
```

```
    print(name)
```

```
    print(age)
```

```
    print(gender)
```

① Student can have nearly 10 properties.

Structure

```
Struct Student
```

```
    String name
```

```
    int age
```

```
    String gender
```

It looks like a class, But

(i) No methods

(ii) It cannot control the visibility / accessibility of the variables.

Something

someone

```
printStudent ( Student st )
```

```
    print(st.name)
```

```
    print(st.age)
```

```
    print(st.gender)
```

```
    |
```

It looks like,

"Something is happening to someone"

v/s

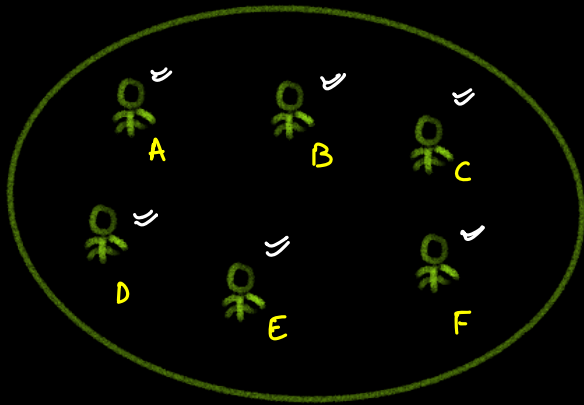
"someone is doing something".

In reality, entities perform actions, whereas in procedural languages

action is performed on entities.

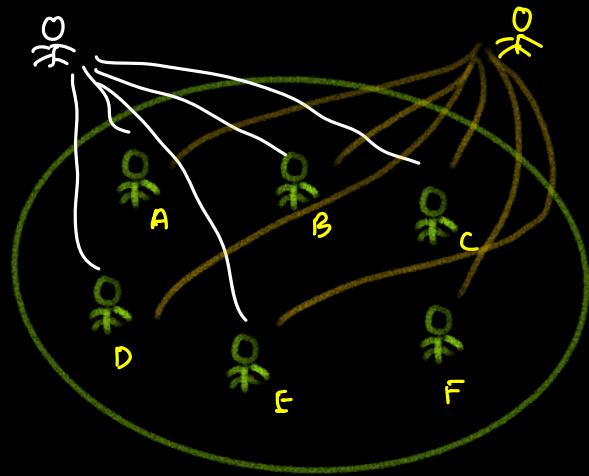
Better version, st.print()

Imagine, there are 6 people, A, B, C, D, E, F.



OOP world.

A, B, C, D, E, F has complete control on them



Procedural world

Anyone can control A, B, C, D, E, F.

OOP

In OOP, software systems consists of entities, Each entity has attributes and behaviour associated with it (it can only be controlled by that particular entity).

class Student

```
String name
int age
String gender
;
void print()
```

Variables are controlled by class itself.

Behaviours are tied to this class.

Cons of procedural prog

- * It's difficult to think in terms of practical entities
- * It's difficult to understand / debug
- * The concept of 'privacy' is missing.
"entities are the puppets".

OOP introduction

The need of thinking software systems just like how humans think, lead to OOP.

- (i) Entities are the core of OOP
- (ii) Every entity has some attribute and behaviour.

Pillars of OOP

- * Abstraction
- * Encapsulation
- * Inheritance
- * Polymorphism.

IMO, we've 1 principle and 3 pillars to support it in OOP.

ex: I will be a good person

- * I'll always be kind
- * I'll be truthful
- * Respect everyone.

Abstraction

"The quality of dealing with ideas".

↓

entities

(i) Anything that has some info

attributes

(ii) Anything that has associated behaviour

methods.

"Abstraction is representing complex software system in terms of ideas, because ideas are easy to understand".

Purpose : Others do not need the details of the idea.

Encapsulation

en-capsul-ation

↓

Why not normal medicine

* It'll hold the medicine powder together

* It protects the medicine from the outside environment.

Almost the same is being served by encapsulation in OOP.

→ What are we trying to hold together? -- attributes and behaviours.

Where to store them together? -- Class

class

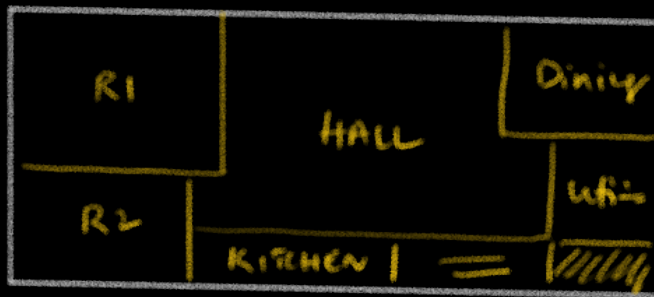
"It's a blueprint of an idea"

You're a real estate builder.

1bhk, 2bhk, 3bhk.



we will blueprint of a 2bhk house



Blueprint of a student

```
class Student
```

```
    int age
```

```
    int rollNo
```

```
    String name
```

```
    double psp
```

```
    float salary
```

```
    change Batch( __ , __ )
```

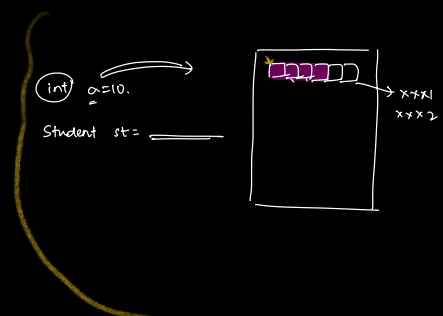
```
    pause course ( __ )
```

```
    extend class ( __ )
```

* A class takes no memory

* Not a real entity

* Multiple instances of the same.



Object

Real instances of a class.

Objects will occupy memory (They are real).

How to create an object.

```
int a = 10;
```

```
Student keerthi = new Student();
```

↓ ↓ ↓
classname [Obj Name] class name